

A Simplified MAL-C Problem: A θ -Threshold Dual-Track Flow-Splitting Routing Algorithm and Its Analysis

Abstract

We study a simplified variant of the *Minimum Aged Labelling with Capacity constraints* (MAL-C) problem. The objective is to assign departure-time labels to the edges of a directed graph and to route a set of freight demands so as to minimize the total number of unit-capacity trucks dispatched, where the flow on each (edge, time) pair must be rounded *up* to a whole truck. We present a dual-track algorithm that routes bulky integer flow along shortest paths and consolidates fractional flow onto a Directed Steiner Forest (DSF) backbone whose departure times are produced by a global *dual-label* construction. Under a store-and-forward model (freight may wait at nodes at no cost), we prove that the labels turn the canonical route of every source–destination pair—within an SCC or across SCCs—into a temporal (time-respecting) path, and that every backbone edge carries at most two distinct departure labels, so that fractional flows from different demands genuinely share trucks. Balancing the two tracks through a splitting threshold θ yields an approximation ratio of $O(\sqrt{h} N^{1/3} + N^{2/3})$, where $N = |V|$ and h is the number of demands.

1 Problem Description

We consider a simplified version of the *Minimum Aged Labelling with Capacity constraints* (MAL-C) problem. To isolate the core mechanisms—topological path selection and the construction of time labels—we drop the original problem’s *maximum-label constraint* and its *demand-release times*.

Input.

- A directed graph $G = (V, E)$ together with an edge travel-time function $w : E \rightarrow \mathbb{N}$. We write $N = |V|$.
- A set of h demands $D = \{d_1, \dots, d_h\}$. Each demand d_i specifies a source $\text{src}(d_i)$, a destination $\text{dst}(d_i)$, and a freight amount $\text{dem}(d_i) \in \mathbb{R}_{\geq 0}$.

Output and objective. The algorithm must, for every edge $e \in E$, decide a set of *departure times* (the label set $\lambda(e)$), and it must route every unit of freight from its source to its destination. Trucks have capacity normalized to 1. Even when the total flow that traverses an edge at a given time is fractional, an entire truck must be dispatched (*unconditional round-up*). Writing $f_i(e, t)$ for the flow of demand i on edge e departing at time t , the goal is, subject to flow conservation and connectivity, to minimize the total number of trucks:

$$\min \sum_{e \in E} \sum_t \left\lceil \sum_{i=1}^h f_i(e, t) \right\rceil. \quad (1)$$

Algorithm 1 θ -Threshold Dual-Track Flow Splitting

Require: Graph $G = (V, E)$, demand set D with $|D| = h$, splitting threshold $\theta \in (0, 1)$.

- 1: Collect the sources and destinations of all demands; compute an approximate **Directed Steiner Forest** on G , obtaining the backbone $E_{\text{SF}} \subseteq E$.
 - 2: $\lambda_{\text{SF}} \leftarrow \text{ASSIGN-DUAL-LABELS}(E_{\text{SF}}, w)$ $\triangleright$ assign departure times to the backbone
 - 3: **for** each demand $d_i \in D$ **do**
 - 4: Split $\text{dem}(d_i)$ into an integer part d_I and a fractional part d_f with $0 \leq d_f < 1$.
 - 5: **Integer routing:** send the d_I flow along a shortest path of G , **with the corresponding labels.**
 - 6: **Fractional routing:**
 - 7: **if** $d_f \leq 2\theta$ **then**
 - 8: route the entire fractional flow d_f on E_{SF} (consolidated routing).
 - 9: **else** $\triangleright d_f > 2\theta$
 - 10: force a flow of size θ onto E_{SF} (consolidated routing);
 - 11: route the remaining $d_f - \theta$ on a shortest path of G .
 - 12: **end if**
 - 13: **end for**
 - 14: **Forced re-routing rule:** every fractional flow placed on E_{SF} must strictly follow the *time-varying simple path* produced by **ASSIGN-DUAL-LABELS**: first travel up the reverse in-tree to a meeting point, then transfer to the forward out-tree.
-

2 Algorithm Design

In a logistics network the largest source of waste is “dispatching a truck that is not full.” Our strategy is a *dual-track* scheme. Bulky integer flow is routed directly along shortest paths (SP) to save travel time, while sparse fractional flow—which would otherwise force every demand to dispatch its own near-empty truck—is consolidated onto a backbone network, a Directed Steiner Forest (DSF), where many demands ride together.

A splitting threshold $\theta \in (0, 1)$ governs how much fractional flow is diverted to the backbone. The main routine (Algorithm 1) calls the label-construction subroutine (Algorithm 2).

To ensure that the fractional flows on the backbone do not produce additional wasteful round-ups merely because their departure times are misaligned, we design the label-construction subroutine so that, by construction, *every backbone edge carries at most two distinct departure labels*.

3 Temporal Feasibility and the Two-Label Guarantee

The benefit of the dual-label construction relies on a single modeling assumption, which is consistent with the objective (1) because that objective charges only for trucks *on edges* and never for freight resting at a node.

Assumption 1 (Free store-and-forward). Freight may be buffered at any node for an arbitrary amount of time at no cost. (This is admissible precisely because the simplified problem omits the maximum-label / aging constraint of full MAL-C.)

The key property of the construction is that each edge’s label is a function of its *position*—its depth in a tree and the base time of its SCC—and is *independent of which demand* traverses it. We make feasibility precise through the notion of a *temporal path* and prove that the labels turn

Algorithm 2 Global Dual-Label Construction

```
1: function ASSIGN-DUAL-LABELS( $E_{\text{SF}}, w$ )
2:   Contract  $E_{\text{SF}}$  into a DAG and obtain its strongly connected components (SCCs)
    $C_1, C_2, \dots, C_K$  in topological order.
3:   for each SCC  $C_i$  do
4:     Choose a center  $h_i$ . Using the travel times  $w(e)$ , build, rooted at  $h_i$ :
5:       a reverse in-tree  $T_{\text{in}}^{(i)}$ , recording the weighted depth  $d_{\text{in}}(\cdot)$  from a node to the root;
6:       a forward out-tree  $T_{\text{out}}^{(i)}$ , recording the weighted depth  $d_{\text{out}}(\cdot)$  from the root to a node.
7:       Let  $D_{\text{in}}^{(i)} = \max_x d_{\text{in}}(x)$  and  $D_{\text{out}}^{(i)} = \max_x d_{\text{out}}(x)$  be the in- and out-radii of  $C_i$ .
8:   end for
9:   Compute a global base time  $T_i$  for each SCC by dynamic programming, so that  $T_i$  exceeds
   the schedule of every earlier SCC and dominates the arrival time of every cross-boundary edge
   entering  $C_i$  (see Lemma 2).
10:  Label assignment. Initialize  $\lambda(e) = \emptyset$  for all  $e \in E_{\text{SF}}$ . For each edge  $e = (x, y) \in E_{\text{SF}}$ :
11:    if  $e \in T_{\text{in}}^{(i)}$ :  $t_{\text{in}}(e) = T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(x)$ ;
12:    if  $e \in T_{\text{out}}^{(i)}$ :  $t_{\text{out}}(e) = T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(x)$ ;
13:    if  $e$  is a cross-boundary edge:  $t_{\text{cross}}(e) = T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)}$ .
14:  return the label set  $\lambda_{\text{SF}}$ .
15: end function
```

the canonical route of *every* terminal pair into one, whether the two endpoints lie in the same SCC or in different SCCs.

Definition 1 (Temporal path / journey). Given the labels produced by ASSIGN-DUAL-LABELS, a *temporal path* (time-respecting journey) from u to v is a walk $u = z_0, z_1, \dots, z_k = v$ in E_{SF} together with chosen departure times $\tau_j \in \lambda((z_{j-1}, z_j))$ that satisfy the time-respecting condition

$$\tau_{j+1} \geq \tau_j + w((z_{j-1}, z_j)) \quad (1 \leq j < k),$$

i.e. every hop departs no earlier than the freight's arrival at its tail. Under Assumption 1 a temporal path is exactly a physically realizable journey: the freight rides edge (z_{j-1}, z_j) at time τ_j and idles at z_j for the non-negative slack $\tau_{j+1} - (\tau_j + w)$.

Definition 2 (Canonical backbone route). Let u, v be terminals joined through the backbone, and let $C_{i_0} \prec C_{i_1} \prec \dots \prec C_{i_m}$ be the SCCs visited in topological order, with $C_{i_0} = C(u)$ the SCC of u and $C_{i_m} = C(v)$ that of v , where consecutive SCCs $C_{i_{l-1}}, C_{i_l}$ are joined by a cross edge $c_l = (p_l, q_l)$, $p_l \in C_{i_{l-1}}, q_l \in C_{i_l}$ ($1 \leq l \leq m$). The *canonical route* $R(u, v)$ obeys the forced re-routing rule inside every visited SCC: it enters at the entry node (u when $l = 0$, otherwise q_l), climbs the in-tree until the first vertex that also lies in the out-tree—the *meeting vertex* μ_l (in the worst case the center h_{i_l})—then descends the out-tree to the exit node (v when $l = m$, otherwise p_{l+1}); the segments are spliced by the cross edges c_l . When $m = 0$ (a single SCC) the route is simply $u \uparrow \mu_0 \downarrow v$.

We prove temporality in three movements—the climb up the in-tree, the *turn* at the meeting vertex, and the descent down the out-tree—then dominate the cross edges and splice everything by induction. The turn is the crux; unlike a unit-weight sketch, the argument below keeps the travel times $w(e)$ explicit, so the waiting slack is *computed* rather than assumed.

Lemma 1 (Each SCC segment is temporal). Let $u, v \in C_i$, and let μ be the meeting vertex: the first vertex on the canonical ascent from u that also belongs to the out-tree $T_{\text{out}}^{(i)}$ (in the worst case

$\mu = h_i$). If the freight is released at u by time $T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(u)$, the segment $u \xrightarrow{T_{\text{in}}^{(i)}} \mu \xrightarrow{T_{\text{out}}^{(i)}} v$ is a temporal path: it reaches μ at $T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(\mu)$, may leave μ on the out-tree at $T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(\mu)$ (a non-negative wait of $d_{\text{in}}(\mu) + d_{\text{out}}(\mu)$), and reaches v at $T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(v)$. The whole segment lives in the window $[T_i, T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)}]$.

Proof. *Climb (in-tree).* Let $u = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_p = \mu$ be the root-ward path with $e_k = (x_k, x_{k+1}) \in T_{\text{in}}^{(i)}$, so $d_{\text{in}}(x_k) = w(e_k) + d_{\text{in}}(x_{k+1})$. Departing x_k at $t_{\text{in}}(e_k) = T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(x_k)$, the freight reaches x_{k+1} at

$$t_{\text{in}}(e_k) + w(e_k) = (T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(x_k)) + w(e_k) = T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(x_{k+1}) = t_{\text{in}}(e_{k+1}),$$

so each climb hop is time-respecting with zero slack and the arrival at μ is $T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(\mu)$.

Turn (meeting vertex). Let a be the in-tree predecessor of μ and b its first out-tree child, so the freight arrives via (a, μ) and leaves via (μ, b) . Accounting for the travel time $w(a, \mu)$, the arrival is $t_{\text{in}}(a, \mu) + w(a, \mu) = T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(\mu)$ while the out-departure is $t_{\text{out}}(\mu, b) = T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(\mu)$; their difference is the waiting slack

$$t_{\text{out}}(\mu, b) - (t_{\text{in}}(a, \mu) + w(a, \mu)) = d_{\text{in}}(\mu) + d_{\text{out}}(\mu) \geq 0.$$

Equivalently, the gap between the two labels is $t_{\text{out}}(\mu, b) - t_{\text{in}}(a, \mu) = d_{\text{in}}(\mu) + d_{\text{out}}(\mu) \geq w(a, \mu)$, i.e. at least the travel time of the incoming edge—which is precisely the weight-aware form of “the out-tree departure follows the in-tree arrival.” Hence the turn is time-respecting.

Descent (out-tree). Let $\mu = y_0 \rightarrow y_1 \rightarrow \dots \rightarrow y_q = v$ with $g_k = (y_k, y_{k+1}) \in T_{\text{out}}^{(i)}$ and $d_{\text{out}}(y_{k+1}) = d_{\text{out}}(y_k) + w(g_k)$. Departing y_k at $t_{\text{out}}(g_k) = T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(y_k)$, the freight reaches y_{k+1} at $T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(y_{k+1}) = t_{\text{out}}(g_{k+1})$: again time-respecting with zero slack, ending at $T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(v) \leq T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)}$. Finally, the earliest departure anywhere in C_i is at the deepest in-tree node, $T_i + D_{\text{in}}^{(i)} - D_{\text{in}}^{(i)} = T_i$, so any entry that arrives by T_i is admissible. $\square$

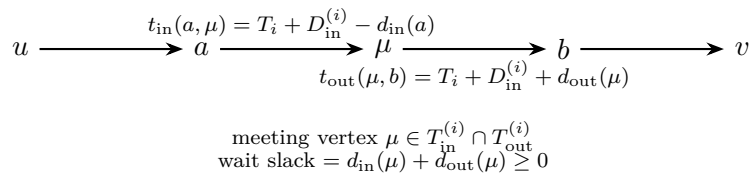


Figure 1: Intra-SCC (Lemma 1). The freight climbs the in-tree to the meeting vertex μ , waits the non-negative slack $d_{\text{in}}(\mu) + d_{\text{out}}(\mu)$, then descends the out-tree. Once the travel time $w(a, \mu)$ is included, the out-tree departure never precedes the in-tree arrival.

For a cross edge the crux is that its label $t_{\text{cross}} = T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)}$ is the *latest* timestamp C_i ever issues, so it **dominates** the arrival at the tail a no matter which tree carried the freight there.

Lemma 2 (Cross edges preserve temporality). *Let $c = (a, b)$ be a cross edge with $a \in C_i$, $b \in C_j$. Then the freight reaches a no later than $t_{\text{cross}}(c) = T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)}$, so it may depart c at its single label. If, in topological order, the base times satisfy*

$$T_j \geq \max_{(a,b): b \in C_j} (t_{\text{cross}}(a, b) + w(a, b)) \quad \text{and} \quad T_j > T_{j-1}$$

(well defined since the contracted graph is a DAG, so the bound depends only on strictly earlier SCCs), then the freight reaches b by T_j and waits for C_j 's schedule. Hence the cross hop and the hand-off into C_j are time-respecting.

Proof. We bound the arrival at a by which tree last carried the freight there.

Case 1 (a reached through the out-tree). If $(x, a) \in T_{\text{out}}^{(i)}$ is the last hop, then by the descent of Lemma 1 the arrival at a is

$$t_{\text{out}}(x, a) + w(x, a) = T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(x) + w(x, a) = T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(a) \leq T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)} = t_{\text{cross}}(c),$$

using $d_{\text{out}}(a) = d_{\text{out}}(x) + w(x, a) \leq D_{\text{out}}^{(i)}$.

Case 2 (a reached through the in-tree). Then the arrival at a is $T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(a)$, and

$$t_{\text{cross}}(c) - (T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(a)) = D_{\text{out}}^{(i)} + d_{\text{in}}(a) \geq 0$$

holds trivially.

In either case the arrival at a is at most $t_{\text{cross}}(c)$, so the freight (waiting the slack if necessary) departs c at $t_{\text{cross}}(c)$ and reaches b at $t_{\text{cross}}(c) + w(c) \leq T_j$. As the in-tree departure at b is $T_j + D_{\text{in}}^{(j)} - d_{\text{in}}(b) \geq T_j$, the freight waits at b and continues up C_j 's in-tree. $\square$

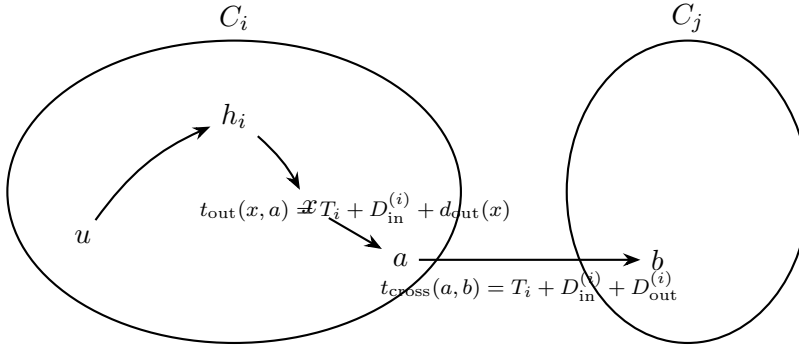


Figure 2: Cross edge, Case 1 (Lemma 2). The tail a is reached by descending the out-tree, arriving at $T_i + D_{\text{in}}^{(i)} + d_{\text{out}}(a) \leq T_i + D_{\text{in}}^{(i)} + D_{\text{out}}^{(i)} = t_{\text{cross}}(a, b)$.

Theorem 1 (Universal temporal realizability). *For every ordered terminal pair (u, v) connected through the backbone—whether u and v lie in the same SCC or in different SCCs—the canonical route $R(u, v)$ is a temporal path under the labels of ASSIGN-DUAL-LABELS. Consequently, under Assumption 1, all backbone demands are simultaneously and physically routable.*

Proof. Induct on the number m of cross edges of $R(u, v)$. *Base $m = 0$:* u, v share an SCC and $R(u, v) = u \uparrow \mu_0 \downarrow v$ is a temporal path by Lemma 1, releasing u at its in-tree label. *Inductive step:* suppose the prefix of $R(u, v)$ up to the entry node q_l of C_{i_l} is a temporal path that delivers the freight to q_l by time T_{i_l} (this holds for $l = 1$ by the base case followed by Lemma 2). The segment inside C_{i_l} from q_l to its exit node is temporal by Lemma 1, since its release time $\geq T_{i_l}$ is admissible; the following cross edge c_{l+1} and the hand-off into $C_{i_{l+1}}$ are temporal by Lemma 2 and deliver the freight to q_{l+1} by $T_{i_{l+1}}$. Splicing time-respecting sub-walks yields a time-respecting walk, so $R(u, v)$ is a temporal path; the case u, v in different SCCs is exactly $m \geq 1$. $\square$

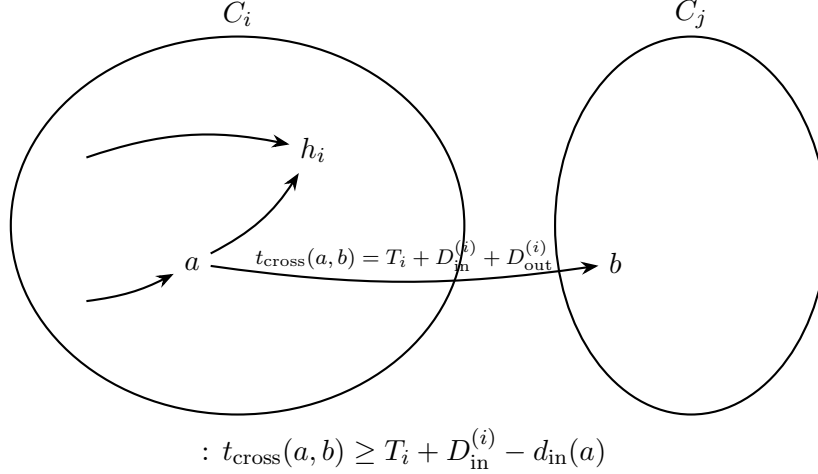


Figure 3: Cross edge, Case 2 (Lemma 2). The tail a lies on the in-tree, arriving at $T_i + D_{\text{in}}^{(i)} - d_{\text{in}}(a)$; since $D_{\text{out}}^{(i)} + d_{\text{in}}(a) \geq 0$ the cross departure trivially dominates it.

Theorem 2 (Two labels per edge and consolidation). *ASSIGN-DUAL-LABELS assigns at most two distinct departure labels to every backbone edge, and under Assumption 1 all fractional flows routed on a common backbone edge depart on those labels. Hence each backbone edge incurs at most two round-ups in the objective (1).*

Proof. Within an SCC the in-tree and the out-tree are arborescences of opposite orientation (toward, respectively away from, the center). A directed edge $e = (x, y)$ may belong to the in-tree—used by up-phase traffic with label $t_{\text{in}}(e)$ —and *simultaneously* to the out-tree—used by down-phase traffic with label $t_{\text{out}}(e)$ —as happens on a directed cycle $h_i \rightarrow x \rightarrow y \rightarrow h_i$, but it plays no third role. A cross-boundary edge is disjoint from the trees and carries the single label $t_{\text{cross}}(e)$. Hence every backbone edge has at most two labels. By Theorem 1 each demand’s canonical route is a temporal path that departs e exactly on the label of the role (t_{in} , t_{out} , or t_{cross}) in which it traverses e ; thus all demands using e in the same role depart together, and the trucks on e number at most $\lceil f_{t_1}(e) \rceil + \lceil f_{t_2}(e) \rceil$, i.e. at most two round-ups. $\square$

4 Approximation-Ratio Analysis

We compare the algorithm’s cost $\text{ALG} = \text{ALG}_{\text{SP}} + \text{ALG}_{\text{SF}}$ against the true optimum OPT of (1). Although OPT is unknown, it is bounded below by two ideal quantities.

Definition 3 (Lower bounds).

- **Fractional-flow bound** OPT_{frac} : the cost when fractional trucks are permitted (no round-up) and all flow travels along shortest paths, $\text{OPT}_{\text{frac}} = \sum_{i=1}^h \text{dem}(d_i) \cdot \text{len}(\text{SP}_i)$, where $\text{len}(\text{SP}_i)$ is the number of edges on a shortest $\text{src}(d_i)$ – $\text{dst}(d_i)$ path. Since every unit of flow is forced to round up in reality, $\text{OPT} \geq \text{OPT}_{\text{frac}}$.
- **Connectivity bound** OPT_{SF} : regardless of how small a flow is, each source must be physically connected to its destination, so the optimum solution contains a Directed Steiner Forest; with unit edge cost, the minimum DSF edge count is a floor, $\text{OPT} \geq \text{OPT}_{\text{SF}}$.

Throughout, $\rho_{\text{DSF}} = O(N^{2/3})$ denotes the approximation factor of the DSF routine used in line 1 of Algorithm 1, so the backbone satisfies $|E_{\text{SF}}| \leq \rho_{\text{DSF}} \cdot \text{OPT}_{\text{SF}} = O(N^{2/3}) \cdot \text{OPT}_{\text{SF}}$.

4.1 Cost of the shortest-path (SP) track

The SP track carries (i) the integer parts d_I of every demand and (ii) the leftover fractional pieces $x = d_f - \theta$ for demands with $d_f > 2\theta$.

Integer flow. Integer flow never wastes a truck: on any edge the rounded value of a sum of integers equals the sum itself, $\lceil \sum d_I \rceil = \sum d_I$. Hence

$$\text{ALG}_{\text{SP}}^{\text{int}} = \sum_{i=1}^h d_I^{(i)} \cdot \text{len}(\text{SP}_i) \leq \sum_{i=1}^h \text{dem}(d_i) \cdot \text{len}(\text{SP}_i) = \text{OPT}_{\text{frac}}. \quad (2)$$

Leftover fractional flow. By the splitting rule, a fractional piece reaches the SP track only when $d_f > 2\theta$, in which case its size satisfies the key inequality

$$x = d_f - \theta > 2\theta - \theta = \theta. \quad (3)$$

On the SP track each such piece is rounded up to $\lceil x \rceil = 1$ truck per edge. Using $x > \theta \Rightarrow 1 < x/\theta$ from (3),

$$\text{ALG}_{\text{SP}}^{\text{frac}} = \sum_{i: d_f > 2\theta} 1 \cdot \text{len}(\text{SP}_i) < \sum_{i: d_f > 2\theta} \frac{x_i}{\theta} \text{len}(\text{SP}_i) = \frac{1}{\theta} \sum_{i: d_f > 2\theta} x_i \text{len}(\text{SP}_i) \leq \frac{1}{\theta} \text{OPT}_{\text{frac}}, \quad (4)$$

where the last step uses $x_i \leq \text{dem}(d_i)$ and that the pieces $\{x_i\}$ form a sub-collection of the total flow, so $\sum x_i \text{len}(\text{SP}_i) \leq \sum_i \text{dem}(d_i) \text{len}(\text{SP}_i) = \text{OPT}_{\text{frac}}$. (This is the corrected form of the bound: the relation to OPT_{frac} is an *inequality*, not an equality, and the integer-flow contribution (2) is accounted for separately.)

Adding (2) and (4),

$$\boxed{\text{ALG}_{\text{SP}} \leq \left(1 + \frac{1}{\theta}\right) \text{OPT}_{\text{frac}} = O\left(\frac{1}{\theta}\right) \cdot \text{OPT}.} \quad (5)$$

Intuitively: the smaller θ is, the lower the threshold for diverting flow to the SP track, and the larger the round-up blow-up factor $1/\theta$.

4.2 Cost of the Steiner-forest (SF) track

By the splitting rule, every demand injects at most 2θ of flow into the backbone (it injects $d_f \leq 2\theta$ when $d_f \leq 2\theta$, and exactly θ when $d_f > 2\theta$). Let demand i route its backbone flow $s_i \leq 2\theta$ along a path $R_i \subseteq E_{\text{SF}}$, and let $F_e = \sum_{i: e \in R_i} s_i$ be the total flow on backbone edge e .

By Theorem 2 each backbone edge carries at most two labels with flows $f_{t_1}(e), f_{t_2}(e)$ summing to F_e . Since $\lceil z \rceil \leq z + 1$ for $z \geq 0$, the truck count on a single edge obeys

$$\lceil f_{t_1}(e) \rceil + \lceil f_{t_2}(e) \rceil \leq F_e + 2. \quad (6)$$

Summing (6) over all backbone edges,

$$\text{ALG}_{\text{SF}} \leq \sum_{e \in E_{\text{SF}}} (F_e + 2) = \underbrace{\sum_{e \in E_{\text{SF}}} F_e}_{\text{flow term}} + \underbrace{2|E_{\text{SF}}|}_{\text{infrastructure term}}. \quad (7)$$

Flow term. Each backbone path has at most $|E_{\text{SF}}|$ edges, so

$$\sum_{e \in E_{\text{SF}}} F_e = \sum_{i=1}^h s_i |R_i| \leq 2\theta \sum_{i=1}^h |R_i| \leq 2\theta \cdot h \cdot |E_{\text{SF}}| = 2h\theta |E_{\text{SF}}|. \quad (8)$$

Infrastructure term. Even an edge that carries only an infinitesimal flow must dispatch a whole truck per label; with at most two labels this is at most 2 trucks per edge, giving the $2|E_{\text{SF}}|$ term. *This term is independent of θ and must not be dropped:* it is the unavoidable cost of physically operating the backbone.

Combining (7)–(8) and $|E_{\text{SF}}| \leq O(N^{2/3})\text{OPT}_{\text{SF}}$,

$$\boxed{\text{ALG}_{\text{SF}} \leq 2(h\theta + 1) |E_{\text{SF}}| = O((h\theta + 1) N^{2/3}) \cdot \text{OPT}_{\text{SF}} \leq O((h\theta + 1) N^{2/3}) \cdot \text{OPT}.} \quad (9)$$

Intuitively: the larger θ is, the more total flow you allow into the backbone and the more the flow term grows; the additive $N^{2/3}$ is the price of building the backbone at all.

4.3 Balancing the bounds and the final ratio

Adding (5) and (9), the overall ratio is

$$\boxed{\text{TotalRatio}(\theta)} = O\left(\frac{1}{\theta}\right) + O(h\theta N^{2/3}) + O(N^{2/3}). \quad (10)$$

Only the first two terms depend on θ . Minimizing $\frac{1}{\theta} + h\theta N^{2/3}$ by equating the two,

$$\frac{1}{\theta} = h\theta N^{2/3} \implies \theta^2 = \frac{1}{h N^{2/3}} \implies \theta^* = \frac{1}{\sqrt{h} N^{1/3}} \in (0, 1). \quad (11)$$

At θ^* each of the first two terms equals $\sqrt{h} N^{1/3}$, while the third is unchanged, so

$$\boxed{\text{TotalRatio}(\theta^*) = O(\sqrt{h} N^{1/3} + N^{2/3}) = O(\max\{\sqrt{h} N^{1/3}, N^{2/3}\}).} \quad (12)$$

Theorem 3 (Approximation guarantee). *Under Assumption 1, Algorithm 1 with threshold $\theta^* = 1/(\sqrt{h} N^{1/3})$ produces a feasible routing and labelling whose cost is within a factor $O(\sqrt{h} N^{1/3} + N^{2/3})$ of the optimum of (1).*

Remark 1 (Two regimes). The bound (12) separates into two regimes, with the crossover at $h = N^{2/3}$:

$$\text{TotalRatio}(\theta^*) = \begin{cases} O(\sqrt{h} N^{1/3}), & h \geq N^{2/3} \text{ (round-up dominated)}, \\ O(N^{2/3}), & h < N^{2/3} \text{ (backbone dominated)}. \end{cases}$$

When demands are plentiful the dual-track splitting is what limits the cost; when demands are few the cost is governed by the cost of the Directed Steiner Forest itself, i.e. by the DSF approximation factor $\rho_{\text{DSF}} = O(N^{2/3})$. No choice of θ removes the $N^{2/3}$ term, since the backbone must be built to enable consolidation.

Remark 2 (On a common pitfall). A naive analysis drops the infrastructure term $2|E_{\text{SF}}|$ in (7), folding it into $O(h\theta N^{2/3})$. That is valid only when $h\theta \geq 1$. At the balancing optimum, however, $h\theta^* = \sqrt{h}/N^{1/3} < 1$ whenever $h < N^{2/3}$, so the additive term in fact *dominates* and cannot be absorbed. Retaining it yields the honest bound (12); omitting it would incorrectly claim $O(\sqrt{h} N^{1/3})$.

5 Conclusion

We presented a dual-track routing-and-labelling algorithm for the simplified MAL-C problem. The θ -threshold rule sends bulky integer flow along shortest paths and consolidates fractional flow onto a Directed Steiner Forest backbone. The global dual-label construction equips the backbone with a temporally consistent schedule: under a free store-and-forward model the canonical route of every terminal pair is a temporal path (Theorem 1)—intra-component segments are exactly synchronized and cross-component hops are absorbed by base times that dominate all incoming arrivals—so that each backbone edge carries at most two labels (Theorem 2) and fractional flows from different demands genuinely share trucks. A careful, complete cost analysis—retaining the θ -independent backbone term—gives an approximation ratio of

$$O(\sqrt{h} N^{1/3} + N^{2/3}),$$

attained at $\theta^* = 1/(\sqrt{h} N^{1/3})$. The bound is round-up dominated when $h \geq N^{2/3}$ and backbone dominated otherwise, the latter reflecting the intrinsic cost of the Directed Steiner Forest approximation.